JavaScript

From Beginner to Professional

A Complete Student Textbook

Learn by Building Real-World Projects

Welcome to JavaScript

This textbook transforms you from a complete beginner to a confident JavaScript developer. Every concept is taught with real-world context and runnable code you can try immediately in your browser's console or any HTML file.

JavaScript is the world's most popular programming language — it runs on every website, powers servers via Node.js, and is the language of the modern web. By the end of this book, you'll be able to build interactive web pages, games, and dynamic applications.

How to Use This Book

- Read each concept, then immediately run the code examples
- Open your browser's DevTools (F12) and use the Console tab to experiment
- Complete the exercises and projects — they are where real learning happens
- Look up the 'Try It' boxes for practical, real-world context

Setting Up Your Environment

You need two things: a code editor and a web browser.

- Editor: Download VS Code (free) from code.visualstudio.com
- Browser: Chrome or Firefox — both have excellent developer tools

Your First Test

Open Chrome, press F12, click Console, and type this:

```
console.log("Hello, JavaScript World!");
// Output: Hello, JavaScript World!
```

If you see the text printed, you're ready. That's your first JavaScript program!

💡 TIP

Bookmark: https://developer.mozilla.org/en-US/docs/Web/JavaScript — MDN is the official JavaScript reference. Use it constantly.

CHAPTER 1

Getting Started

Variables, Data Types & Operators

Variables — Storing Data

A variable is a named container for storing a value. JavaScript has three ways to declare variables:

```
// let — block-scoped, can be reassigned
let userName = 'Alice';
userName = 'Bob';  // OK

// const — block-scoped, CANNOT be reassigned
const PI = 3.14159;
// PI = 3;  // ERROR!

// var — old way, function-scoped (avoid in modern JS)
var age = 25;
```

📏 Naming Rules

- ✓ Use camelCase: firstName, totalPrice, isLoggedIn
- ✓ Start with a letter, underscore, or $
- ✗ Cannot start with a number: 1name (invalid)
- ✗ Cannot use reserved words: let, class, return

Primitive Data Types

JavaScript has 7 primitive types. These are the basic building blocks:

```
// STRING — text
let greeting = "Hello, World!";
let name = 'Alice';
let template = `Hello, ${name}!`;  // Template literal

// NUMBER — integers and decimals
let score = 100;
let price = 9.99;
let negative = -50;

// BOOLEAN — true or false
let isLoggedIn = true;
let hasPermission = false;

// NULL — intentional empty value
let selectedUser = null;

// UNDEFINED — declared but not assigned
```

```
let address;
console.log(address);  // undefined

// BIGINT — very large integers
let bigNumber = 9007199254740991n;

// SYMBOL — unique identifiers
let id = Symbol('id');
```

Checking and Converting Types

```
let value = '42';
console.log(typeof value);     // 'string'

// Convert string to number
let num = Number(value);       // 42
let num2 = parseInt('42px');   // 42
let num3 = parseFloat('3.14'); // 3.14

// Convert number to string
let str = String(42);          // '42'
let str2 = (42).toString();    // '42'

// Convert to boolean
console.log(Boolean(0));       // false
console.log(Boolean(''));      // false
console.log(Boolean('hello')); // true
console.log(Boolean(42));      // true
```

🚀 TRY IT — REAL WORLD EXAMPLE

Real-World: A shopping cart stores a product name (string), price (number), and whether it's in stock (boolean). Use the right types!

Operators

Arithmetic Operators

```
let a = 10, b = 3;

console.log(a + b);   // 13  — addition
console.log(a - b);   // 7   — subtraction
console.log(a * b);   // 30  — multiplication
console.log(a / b);   // 3.333... — division
console.log(a % b);   // 1   — modulus (remainder)
console.log(a ** b);  // 1000 — exponentiation (10^3)

// Increment and Decrement
let count = 5;
count++;              // count = 6
count--;              // count = 5
```

```
console.log(++count); // 6 (pre-increment, returns new value)
console.log(count++); // 6 (post-increment, returns old value)
```

Comparison Operators

```
console.log(5 == '5');   // true  (loose equality — converts type)
console.log(5 === '5');  // false (strict equality — checks type too)
console.log(5 !== 3);   // true
console.log(10 > 5);    // true
console.log(10 <= 10);  // true

// ALWAYS use === for comparisons to avoid bugs!
let userInput = "5";
let expectedValue = 5;
if (userInput === expectedValue) {
  console.log("Match!");
} else {
  console.log("No match — different types");  // This runs
}
```

Logical Operators

```
let isAdult = true;
let hasID = false;

// AND — both must be true
console.log(isAdult && hasID);  // false

// OR — at least one must be true
console.log(isAdult || hasID);  // true

// NOT — flips the value
console.log(!isAdult);          // false

// Real-world: Can user access the bar?
let age = 21;
let hasValidID = true;
let canEnter = age >= 21 && hasValidID;
console.log(canEnter);  // true
```

Assignment Operators

```
let score = 100;
score += 50;   // score = 150  (same as score = score + 50)
score -= 20;   // score = 130
score *= 2;    // score = 260
score /= 4;    // score = 65
score **= 2;   // score = 4225
score %= 100;  // score = 25
```

📝 Chapter Project: Unit Converter

Build a miles-to-kilometers and BMI calculator using what you've learned:

```
// Miles to Kilometers Converter
const MILES_TO_KM = 1.60934;

let miles = 26.2;  // Marathon distance
let kilometers = miles * MILES_TO_KM;
console.log(`${miles} miles = ${kilometers.toFixed(2)} km`);
// Output: 26.2 miles = 42.16 km

// BMI Calculator
let weightKg = 70;
let heightM = 1.75;
let bmi = weightKg / (heightM ** 2);
console.log(`BMI: ${bmi.toFixed(1)}`);  // BMI: 22.9

// BMI Categories
let category = bmi < 18.5 ? 'Underweight' :
               bmi < 25 ? 'Normal' :
               bmi < 30 ? 'Overweight' : 'Obese';
console.log(`Category: ${category}`);  // Normal
```

CHAPTER 2

Arrays & Objects

Working with Collections of Data

Arrays — Ordered Lists

An array stores multiple values in a single variable. Values are ordered by index, starting at 0.

```
// Creating arrays
let fruits = ['apple', 'banana', 'cherry'];
let numbers = [1, 2, 3, 4, 5];
let mixed = ['Alice', 30, true, null];  // Can mix types

// Accessing elements (0-indexed)
console.log(fruits[0]);   // 'apple'
console.log(fruits[2]);   // 'cherry'
console.log(fruits[fruits.length - 1]);  // last: 'cherry'

// Modifying elements
fruits[1] = 'mango';
console.log(fruits);  // ['apple', 'mango', 'cherry']
```

Essential Array Methods

```
let cart = ['shirt', 'pants'];

// Adding items
cart.push('shoes');       // Add to end
cart.unshift('hat');      // Add to start
console.log(cart);  // ['hat', 'shirt', 'pants', 'shoes']

// Removing items
cart.pop();               // Remove from end → returns 'shoes'
cart.shift();             // Remove from start → returns 'hat'
console.log(cart);  // ['shirt', 'pants']

// Finding items
let idx = cart.indexOf('pants');  // 1
let found = cart.includes('shirt');  // true

// Slicing (non-destructive)
let sub = cart.slice(0, 1);  // ['shirt'] — original unchanged

// Sorting
let scores = [40, 100, 1, 5, 25];
scores.sort((a, b) => a - b);  // Ascending
console.log(scores);  // [1, 5, 25, 40, 100]
```

Powerful Array Methods (map, filter, reduce)

```
let prices = [10, 25, 5, 50, 30];

// map — transform every element
let withTax = prices.map(price => price * 1.1);
console.log(withTax);  // [11, 27.5, 5.5, 55, 33]

// filter — keep elements that pass a test
let affordable = prices.filter(price => price <= 25);
console.log(affordable);  // [10, 25, 5]

// reduce — combine all elements into one value
let total = prices.reduce((sum, price) => sum + price, 0);
console.log(total);  // 120

// Chaining — filter then sum
let affordableTotal = prices
  .filter(p => p <= 25)
  .reduce((sum, p) => sum + p, 0);
console.log(affordableTotal);  // 40
```

🚀 TRY IT — REAL WORLD EXAMPLE

Real-World: E-commerce carts use filter() to show items by category, map() to apply discounts, and reduce() to calculate totals.

Multidimensional Arrays

```
// A 3x3 grid (tic-tac-toe board)
let board = [
  ['X', 'O', 'X'],
  ['O', 'X', 'O'],
  ['X', '', 'O']
];

// Access: row 1, col 2
console.log(board[1][2]);  // 'O'

// Month sales data
let salesData = [
  [1200, 1500, 1100],  // Q1
  [1800, 2000, 1700],  // Q2
];
console.log(`Q1 Jan: $${salesData[0][0]}`);  // $1200
```

Objects — Key-Value Pairs

Objects store named properties. Unlike arrays, you access values by key name, not index.

```
// Creating an object
```

```
let user = {
  firstName: 'Alice',
  lastName: 'Johnson',
  age: 28,
  isActive: true,
  address: {
    city: 'New York',
    country: 'USA'
  }
};

// Accessing properties
console.log(user.firstName);         // 'Alice' (dot notation)
console.log(user['age']);            // 28 (bracket notation)
console.log(user.address.city);      // 'New York' (nested)

// Modifying / Adding properties
user.email = 'alice@example.com';    // Add new property
user.age = 29;                       // Update existing

// Deleting
delete user.isActive;
```

Objects in Arrays — The Classic Pattern

```
// Product catalog (array of objects)
let products = [
  { id: 1, name: 'Laptop', price: 999, inStock: true },
  { id: 2, name: 'Mouse',  price: 29,  inStock: true },
  { id: 3, name: 'Webcam', price: 79,  inStock: false }
];

// Find a product
let laptop = products.find(p => p.id === 1);
console.log(laptop.name);  // 'Laptop'

// Get only in-stock products
let available = products.filter(p => p.inStock);
console.log(available.length);  // 2

// Total value of inventory
let totalValue = products.reduce((sum, p) => sum + p.price, 0);
console.log(`Total: $${totalValue}`);  // $1107
```

Destructuring

```
// Array destructuring
let [first, second, ...rest] = [10, 20, 30, 40, 50];
console.log(first);  // 10
console.log(rest);   // [30, 40, 50]

// Object destructuring
let { name, price, category = 'General' } = { name: 'Phone', price: 699 };
```

```
console.log(name);     // 'Phone'
console.log(category); // 'General' (default)

// Function parameter destructuring
function displayProduct({ name, price }) {
  return `${name} costs $${price}`;
}
console.log(displayProduct({ name: 'Phone', price: 699 }));  // Phone costs $699
```

Spread & Rest Operators

```
// Spread: expand array or object
let nums1 = [1, 2, 3];
let nums2 = [4, 5, 6];
let combined = [...nums1, ...nums2];  // [1, 2, 3, 4, 5, 6]

// Copy an object (avoids mutation)
let original = { a: 1, b: 2 };
let copy = { ...original, c: 3 };  // { a: 1, b: 2, c: 3 }

// Merge objects
let defaults = { theme: 'light', lang: 'en', fontSize: 14 };
let userPrefs = { theme: 'dark', fontSize: 18 };
let settings = { ...defaults, ...userPrefs };
// { theme: 'dark', lang: 'en', fontSize: 18 } — userPrefs wins
```

📝 Chapter Project: Product Catalog

```
let catalog = [
  { id: 1, name: 'Laptop',   price: 999,  category: 'Electronics', stock: 15 },
  { id: 2, name: 'Desk',     price: 250,  category: 'Furniture',   stock: 8  },
  { id: 3, name: 'Monitor',  price: 349,  category: 'Electronics', stock: 22 },
  { id: 4, name: 'Chair',    price: 399,  category: 'Furniture',   stock: 5  },
  { id: 5, name: 'Keyboard', price: 89,   category: 'Electronics', stock: 50 },
];

// Get all electronics, sorted by price
let electronics = catalog
  .filter(p => p.category === 'Electronics')
  .sort((a, b) => a.price - b.price);

// Display formatted
electronics.forEach(p => {
  console.log(`${p.name}: $${p.price} (${p.stock} in stock)`);
});
// Keyboard: $89 (50 in stock)
// Monitor: $349 (22 in stock)
// Laptop: $999 (15 in stock)

// Total electronics value
let totalValue = electronics.reduce((sum, p) => sum + (p.price * p.stock), 0);
console.log(`Inventory value: $${totalValue}`);  // $38,463
```

CHAPTER 3

Control Flow

If/Else, Switch & Loops

Conditional Statements

Control flow lets your program make decisions and execute different code based on conditions.

```
// Basic if/else
let temperature = 22;

if (temperature > 30) {
  console.log('Hot — wear shorts!');
} else if (temperature > 20) {
  console.log('Warm — light jacket maybe');  // This runs
} else if (temperature > 10) {
  console.log('Cool — wear a jacket');
} else {
  console.log('Cold — bundle up!');
}

// Nested conditions
let isRaining = false;
if (temperature > 20) {
  if (!isRaining) {
    console.log('Great day for a walk!');
  }
}
```

Ternary Operator — Compact if/else

```
// Syntax: condition ? valueIfTrue : valueIfFalse
let age = 20;
let status = age >= 18 ? 'Adult' : 'Minor';
console.log(status);  // 'Adult'

// Inline in template literal
let score = 75;
let grade = score >= 90 ? 'A' :
            score >= 80 ? 'B' :
            score >= 70 ? 'C' :
            score >= 60 ? 'D' : 'F';
console.log(`Grade: ${grade}`);  // Grade: C

// Real-world: Display price with member discount
let isMember = true;
let basePrice = 100;
let finalPrice = isMember ? basePrice * 0.8 : basePrice;
console.log(`Price: $${finalPrice}`);  // Price: $80
```

Switch Statements

```
let day = 'Monday';

switch (day) {
  case 'Monday':
  case 'Tuesday':
  case 'Wednesday':
  case 'Thursday':
  case 'Friday':
    console.log('Weekday — work time!');
    break;
  case 'Saturday':
  case 'Sunday':
    console.log('Weekend — relax!');
    break;
  default:
    console.log('Invalid day');
}

// Real-world: HTTP status codes
function getStatusMessage(code) {
  switch (code) {
    case 200: return 'OK — success';
    case 404: return 'Not Found';
    case 500: return 'Internal Server Error';
    default:  return `Unknown status: ${code}`;
  }
}
console.log(getStatusMessage(404));  // Not Found
```

Loops

while Loop — when you don't know how many iterations

```
// while — runs while condition is true
let balance = 1000;
let month = 0;

while (balance > 0) {
  balance -= 150;  // Monthly expense
  month++;
  if (month > 20) break;  // Safety: avoid infinite loop
}
console.log(`Balance runs out in ${month} months`);

// do...while — runs at least ONCE
let input;
let attempts = 0;
do {
  input = 'correct';  // Simulating user input
  attempts++;
} while (input !== 'correct');
console.log(`Got correct input in ${attempts} attempt(s)`);
```

for Loop — when you know the count

```
// Classic for loop
for (let i = 0; i < 5; i++) {
  console.log(`Step ${i + 1}`);
}

// Count down
for (let i = 10; i >= 0; i--) {
  process.stdout.write(`${i} `);
}
console.log('Liftoff!');

// Loop through an array
let students = ['Alice', 'Bob', 'Carol'];
for (let i = 0; i < students.length; i++) {
  console.log(`Student ${i + 1}: ${students[i]}`);
}
```

for...of and for...in

```
// for...of — iterate VALUES of iterable
let colors = ['red', 'green', 'blue'];
for (let color of colors) {
  console.log(color);  // red, green, blue
}

// for...in — iterate KEYS of object
let person = { name: 'Alice', age: 28, city: 'NYC' };
for (let key in person) {
  console.log(`${key}: ${person[key]}`);
}
// name: Alice
// age: 28
// city: NYC

// forEach — array method with callback
let prices = [10, 25, 50];
prices.forEach((price, index) => {
  console.log(`Item ${index + 1}: $${price}`);
});
```

Nested Loops

```
// Multiplication table
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    process.stdout.write(`${i * j}\t`);
  }
  console.log();
}
// 1  2  3
// 2  4  6
// 3  6  9
```

```
// break and continue
for (let i = 0; i < 10; i++) {
  if (i === 3) continue;  // Skip 3
  if (i === 7) break;     // Stop at 7
  console.log(i);  // 0, 1, 2, 4, 5, 6
}
```

🚀 TRY IT — REAL WORLD EXAMPLE

Real-World: An online store processes orders in a loop, checks each item's stock (if/else), applies discounts (ternary), and breaks out when a sold-out item is found.

📝 Chapter Project: Rock, Paper, Scissors

```
function rockPaperScissors(playerMove) {
  const moves = ['rock', 'paper', 'scissors'];
  const computerMove = moves[Math.floor(Math.random() * 3)];

  console.log(`You: ${playerMove} | Computer: ${computerMove}`);

  if (playerMove === computerMove) {
    return 'Tie!';
  }

  const wins = {
    rock: 'scissors',
    paper: 'rock',
    scissors: 'paper'
  };

  return wins[playerMove] === computerMove ? 'You win! 🎉' : 'Computer wins! 🤖';
}

// Play 3 rounds
let playerScore = 0, computerScore = 0;
let playerMoves = ['rock', 'scissors', 'paper'];

for (let move of playerMoves) {
  let result = rockPaperScissors(move);
  console.log(result);
  if (result.includes('You win')) playerScore++;
  else if (result.includes('Computer')) computerScore++;
}
console.log(`Final: You ${playerScore} - ${computerScore} Computer`);
```

CHAPTER 4

Functions

Reusable Code Blocks

Function Basics

Functions are reusable blocks of code. They help avoid repetition and make programs organized and maintainable.

```
// Function declaration (hoisted — can be called before definition)
function greet(name) {
  return `Hello, ${name}!`;
}
console.log(greet('Alice'));  // Hello, Alice!

// Function expression (not hoisted)
const multiply = function(a, b) {
  return a * b;
};
console.log(multiply(4, 5));  // 20

// Multiple parameters and default values
function createUser(name, role = 'user', active = true) {
  return { name, role, active };
}
console.log(createUser('Alice'));
// { name: 'Alice', role: 'user', active: true }
console.log(createUser('Bob', 'admin'));
// { name: 'Bob', role: 'admin', active: true }
```

Arrow Functions

Arrow functions are a concise syntax introduced in ES6. They're especially common in modern JavaScript.

```
// Traditional function
function square(n) { return n * n; }

// Arrow function — equivalent
const square2 = (n) => { return n * n; };

// Shorter: single expression — implicit return
const square3 = n => n * n;

// No parameters
const sayHello = () => 'Hello!';

// Multiple parameters
const add = (a, b) => a + b;
```

```
// Common in array methods
let numbers = [1, 2, 3, 4, 5];
let evens = numbers.filter(n => n % 2 === 0);   // [2, 4]
let doubled = numbers.map(n => n * 2);           // [2, 4, 6, 8, 10]
```

Scope — Where Variables Live

```
// Global scope — accessible everywhere
let globalVar = 'I am global';

function myFunction() {
  // Local scope — only inside function
  let localVar = 'I am local';
  console.log(globalVar);  // OK — can access global
  console.log(localVar);   // OK
}
myFunction();
// console.log(localVar);  // ERROR! localVar is not defined

// Block scope with let/const
if (true) {
  let blockVar = 'block scoped';
  var notBlockScoped = 'function scoped';
}
// console.log(blockVar);     // ERROR with let
console.log(notBlockScoped); // OK with var (use let/const instead!)
```

Special Function Patterns

Rest Parameters & Spread in Functions

```
// Rest: collect remaining args into array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3));       // 6
console.log(sum(10, 20, 30, 40)); // 100

// Spread: pass array as individual args
let scores = [88, 92, 45, 67, 99];
console.log(Math.max(...scores));  // 99
console.log(Math.min(...scores));  // 45
```

Callback Functions

```
// A function that accepts another function as argument
function processData(data, callback) {
  console.log('Processing...');
  return callback(data);
}
```

```
let result = processData(42, n => n * 2);
console.log(result);  // 84

// Real-world: setTimeout uses a callback
console.log('Start');
setTimeout(() => {
  console.log('This runs after 2 seconds');
}, 2000);
console.log('End (printed before the timeout!)');
```

Immediately Invoked Function Expression (IIFE)

```
// Runs immediately — scope is private
(function() {
  let privateVar = 'You cannot access me outside!';
  console.log('IIFE running!', privateVar);
})();

// Arrow IIFE
const result = (() => {
  let x = 10;
  let y = 20;
  return x + y;
})();
console.log(result);  // 30
```

Recursive Functions

```
// Function that calls itself
function factorial(n) {
  if (n <= 1) return 1;       // Base case — stops recursion
  return n * factorial(n - 1); // Recursive case
}
console.log(factorial(5));  // 120 (5 × 4 × 3 × 2 × 1)

// Real-world: Traverse folder structure
function listFiles(folder, depth = 0) {
  console.log('  '.repeat(depth) + folder.name);
  if (folder.children) {
    folder.children.forEach(child => listFiles(child, depth + 1));
  }
}
let fileSystem = {
  name: 'src',
  children: [
    { name: 'index.js' },
    { name: 'utils', children: [{ name: 'helper.js' }] }
  ]
};
listFiles(fileSystem);
// src
//   index.js
//   utils
//     helper.js
```

Closures

```
// A function that 'remembers' its outer scope
function makeCounter(start = 0) {
  let count = start;  // Private to makeCounter
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count
  };
}

let counter1 = makeCounter(0);
let counter2 = makeCounter(100);

counter1.increment();
counter1.increment();
counter2.decrement();

console.log(counter1.value());  // 2
console.log(counter2.value());  // 99
// counter1 and counter2 are independent!
```

💡 TIP

Closures are fundamental to JavaScript. They're used in event handlers, module patterns, React hooks, and many design patterns.

📝 Chapter Project: Calculator with History

```
function createCalculator() {
  let history = [];

  function calculate(a, operator, b) {
    let result;
    switch (operator) {
      case '+': result = a + b; break;
      case '-': result = a - b; break;
      case '*': result = a * b; break;
      case '/': result = b !== 0 ? a / b : 'Error: Division by zero'; break;
      default:  result = 'Unknown operator';
    }
    const entry = `${a} ${operator} ${b} = ${result}`;
    history.push(entry);
    return result;
  }

  return {
    calculate,
    getHistory: () => history,
    clearHistory: () => history = []
  };
```

```
}

let calc = createCalculator();
console.log(calc.calculate(10, '+', 5));    // 15
console.log(calc.calculate(20, '*', 3));    // 60
console.log(calc.calculate(100, '/', 4));   // 25
console.log(calc.getHistory());
// ['10 + 5 = 15', '20 * 3 = 60', '100 / 4 = 25']
```

CHAPTER 5

Classes & OOP

Object-Oriented Programming

Object-Oriented Programming

OOP organizes code around objects — real-world entities with properties (data) and methods (behavior). JavaScript uses prototypal inheritance, with ES6 class syntax as a cleaner interface.

```
// Class definition
class Animal {
  constructor(name, sound) {
    this.name = name;    // Instance property
    this.sound = sound;
    this.tricks = [];    // Each animal has own tricks array
  }

  // Method
  speak() {
    return `${this.name} says ${this.sound}!`;
  }

  learnTrick(trick) {
    this.tricks.push(trick);
    return `${this.name} learned: ${trick}`;
  }
}

// Create instances
let dog = new Animal('Rex', 'Woof');
let cat = new Animal('Whiskers', 'Meow');

console.log(dog.speak());         // Rex says Woof!
console.log(cat.speak());         // Whiskers says Meow!
console.log(dog.learnTrick('sit')); // Rex learned: sit
```

Getters and Setters

```
class Product {
  constructor(name, price) {
    this.name = name;
    this._price = price;  // Convention: _ means 'private'
  }

  // Getter — access like a property
  get price() {
    return `$${this._price.toFixed(2)}`;
  }

  // Setter — validate before setting
```

```
  set price(value) {
    if (value < 0) throw new Error('Price cannot be negative');
    this._price = value;
  }

  get discountedPrice() {
    return `$${(this._price * 0.9).toFixed(2)}`;
  }
}

let item = new Product('Laptop', 999.99);
console.log(item.price);             // $999.99
console.log(item.discountedPrice);   // $899.99
item.price = 1099;
console.log(item.price);             // $1099.00
```

Inheritance — Extending Classes

```
class Vehicle {
  constructor(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
    this.speed = 0;
  }

  accelerate(amount) {
    this.speed += amount;
    return `${this.make} ${this.model}: ${this.speed} mph`;
  }

  brake(amount) {
    this.speed = Math.max(0, this.speed - amount);
    return `Slowing to ${this.speed} mph`;
  }

  toString() {
    return `${this.year} ${this.make} ${this.model}`;
  }
}

// ElectricVehicle extends Vehicle
class ElectricVehicle extends Vehicle {
  constructor(make, model, year, batteryRange) {
    super(make, model, year);  // Call parent constructor!
    this.batteryRange = batteryRange;
    this.charge = 100;         // Percent
  }

  // Override parent method
  accelerate(amount) {
    this.charge -= amount * 0.1;  // Uses battery
    return super.accelerate(amount) + ` (Battery: ${this.charge.toFixed(0)}%)`;
  }
```

```
  // New method
  recharge() {
    this.charge = 100;
    return 'Fully charged!';
  }
}

let car = new Vehicle('Toyota', 'Camry', 2022);
let ev = new ElectricVehicle('Tesla', 'Model 3', 2023, 358);

console.log(car.accelerate(60));  // Toyota Camry: 60 mph
console.log(ev.accelerate(60));   // Tesla Model 3: 60 mph (Battery: 94%)
console.log(ev instanceof Vehicle);  // true (inheritance check)
```

Static Methods & Properties

```
class MathUtils {
  // Static: belongs to class, not instances
  static PI = 3.14159;

  static circleArea(radius) {
    return MathUtils.PI * radius ** 2;
  }

  static clamp(value, min, max) {
    return Math.min(Math.max(value, min), max);
  }
}

// Call directly on class — no 'new' needed
console.log(MathUtils.circleArea(5));   // 78.53975
console.log(MathUtils.clamp(150, 0, 100));  // 100

// Practical: ID counter
class User {
  static count = 0;

  constructor(name) {
    User.count++;
    this.id = User.count;
    this.name = name;
  }
}

new User('Alice');
new User('Bob');
new User('Carol');
console.log(User.count);  // 3
```

📝 Chapter Project: Employee Tracking System

```
class Employee {
```

```
  static employeeCount = 0;

  constructor(name, department, salary) {
    Employee.employeeCount++;
    this.id = Employee.employeeCount;
    this.name = name;
    this.department = department;
    this._salary = salary;
    this.hireDate = new Date().toLocaleDateString();
  }

  get salary() { return `$${this._salary.toLocaleString()}`; }
  set salary(val) {
    if (val > 0) this._salary = val;
    else throw new Error('Salary must be positive');
  }

  givRaise(percent) {
    this._salary *= (1 + percent / 100);
    return `${this.name} now earns ${this.salary}`;
  }

  toString() {
    return `[${this.id}] ${this.name} | ${this.department} | ${this.salary}`;
  }
}

class Manager extends Employee {
  constructor(name, department, salary, teamSize) {
    super(name, department, salary);
    this.teamSize = teamSize;
    this.reports = [];
  }

  addReport(employee) {
    this.reports.push(employee);
    return `${employee.name} now reports to ${this.name}`;
  }
}

let alice = new Employee('Alice', 'Engineering', 85000);
let bob = new Employee('Bob', 'Design', 72000);
let manager = new Manager('Carol', 'Engineering', 120000, 5);

console.log(alice.toString());
console.log(alice.givRaise(10));
console.log(manager.addReport(alice));
console.log(`Total employees: ${Employee.employeeCount}`);
```

CHAPTER 6

Built-In Methods & APIs

JavaScript's Powerful Toolbox

String Methods

Strings have many built-in methods for manipulation. Remember: strings are immutable — methods return new strings.

```
let str = 'Hello, JavaScript World!';

// Case
console.log(str.toUpperCase());  // HELLO, JAVASCRIPT WORLD!
console.log(str.toLowerCase());  // hello, javascript world!

// Searching
console.log(str.includes('JavaScript'));  // true
console.log(str.indexOf('World'));         // 19
console.log(str.startsWith('Hello'));      // true
console.log(str.endsWith('!'));            // true

// Extracting
console.log(str.slice(7, 17));    // 'JavaScript'
console.log(str.substring(7, 17)); // 'JavaScript'

// Replacing
console.log(str.replace('World', 'Universe'));
// 'Hello, JavaScript Universe!'
console.log(str.replaceAll('l', 'L'));
// 'HeLLo, JavaScript WorLd!'

// Splitting and joining
let words = str.split(' ');
console.log(words);  // ['Hello,', 'JavaScript', 'World!']
let rejoined = words.join('-');
console.log(rejoined);  // 'Hello,-JavaScript-World!'

// Trimming whitespace
let messy = '  trim me  ';
console.log(messy.trim());       // 'trim me'
console.log(messy.trimStart());  // 'trim me  '
console.log(messy.trimEnd());    // '  trim me'

// Padding
let invoice = '42';
console.log(invoice.padStart(6, '0'));  // '000042'
```

Number & Math Methods

```
let n = 3.14159;
```

```
// Number formatting
console.log(n.toFixed(2));      // '3.14'
console.log(n.toPrecision(4));  // '3.142'
console.log((1234567).toLocaleString());  // '1,234,567'
console.log((0.5).toLocaleString('en-US', {
  style: 'currency', currency: 'USD'
}));  // '$0.50'

// Math object
console.log(Math.round(4.5));  // 5
console.log(Math.floor(4.9));  // 4
console.log(Math.ceil(4.1));   // 5
console.log(Math.abs(-42));    // 42
console.log(Math.sqrt(144));   // 12
console.log(Math.pow(2, 10));  // 1024
console.log(Math.max(3, 1, 4, 1, 5, 9));  // 9
console.log(Math.min(3, 1, 4, 1, 5, 9));  // 1

// Random numbers
let random = Math.random();              // 0 to 0.999...
let rand1to10 = Math.floor(Math.random() * 10) + 1;
let randInRange = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
console.log(randInRange(1, 100));  // Random int between 1 and 100
```

Date Object

```
// Creating dates
let now = new Date();
let specific = new Date('2024-01-15');
let fromParts = new Date(2024, 0, 15, 12, 30);  // Jan 15, 12:30

// Getting date parts
console.log(now.getFullYear());  // 2024
console.log(now.getMonth());     // 0-11 (0 = January!)
console.log(now.getDate());      // Day of month 1-31
console.log(now.getDay());       // 0-6 (0 = Sunday)
console.log(now.getHours());     // 0-23

// Formatting
console.log(now.toLocaleDateString('en-US'));
// e.g., '1/15/2024'
console.log(now.toLocaleString('en-US', {
  weekday: 'long', year: 'numeric',
  month: 'long', day: 'numeric'
}));  // e.g., 'Monday, January 15, 2024'

// Date arithmetic
let future = new Date();
future.setDate(future.getDate() + 30);  // 30 days from now
let diffMs = future - now;
let diffDays = Math.round(diffMs / (1000 * 60 * 60 * 24));
console.log(`Days until then: ${diffDays}`);
```

Regular Expressions

RegEx is a powerful tool for pattern matching in strings. It's used for validation, search, and replace.

```
// Creating RegEx
let pattern = /hello/i;  // i = case insensitive
console.log(pattern.test('Hello World'));  // true

// Common patterns
let emailRegex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
console.log(emailRegex.test('user@example.com'));  // true
console.log(emailRegex.test('not-an-email'));      // false

let phoneRegex = /^\+?[1-9]\d{1,14}$/;
console.log(phoneRegex.test('+12025550123'));  // true

// Extracting matches
let text = 'Contact us at support@site.com or help@site.org';
let emails = text.match(/[a-zA-Z0-9._]+@[a-zA-Z0-9.]+/g);
console.log(emails);  // ['support@site.com', 'help@site.org']

// Replace with regex
let dirty = 'Hello     World    !';
let clean = dirty.replace(/\s+/g, ' ').trim();
console.log(clean);  // 'Hello World !'
```

JSON — Data Interchange Format

```
// JavaScript Object Notation
let user = {
  name: 'Alice',
  age: 28,
  hobbies: ['coding', 'hiking'],
  address: { city: 'NYC', country: 'USA' }
};

// Serialize: object → JSON string (for sending/storing)
let jsonString = JSON.stringify(user);
console.log(typeof jsonString);  // 'string'
console.log(jsonString);
// {"name":"Alice","age":28,"hobbies":["coding","hiking"],"address":{...}}

// Pretty print
console.log(JSON.stringify(user, null, 2));

// Parse: JSON string → object (after receiving data)
let parsed = JSON.parse(jsonString);
console.log(parsed.name);          // 'Alice'
console.log(parsed.hobbies[0]);    // 'coding'

// Common use: saving settings
let settings = { theme: 'dark', language: 'en' };
localStorage.setItem('settings', JSON.stringify(settings));
let loaded = JSON.parse(localStorage.getItem('settings'));
console.log(loaded.theme);  // 'dark'
```

📝 Chapter Project: Countdown Timer

```
function createCountdown(targetDate) {
  function getTimeLeft() {
    let now = new Date();
    let target = new Date(targetDate);
    let diff = target - now;  // milliseconds

    if (diff <= 0) return { expired: true };

    let days = Math.floor(diff / (1000 * 60 * 60 * 24));
    let hours = Math.floor((diff % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
    let minutes = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60));
    let seconds = Math.floor((diff % (1000 * 60)) / 1000);

    return { days, hours, minutes, seconds, expired: false };
  }

  function display() {
    let t = getTimeLeft();
    if (t.expired) {
      console.log('Event has passed!');
      return;
    }
    console.log(
      `${t.days}d ${String(t.hours).padStart(2,'0')}h ` +
      `${String(t.minutes).padStart(2,'0')}m ${String(t.seconds).padStart(2,'0')}s`
    );
  }

  return { getTimeLeft, display };
}

let countdown = createCountdown('2025-12-31');
countdown.display();  // e.g., '365d 10h 30m 45s'
```

CHAPTER 7

The DOM

Manipulating Web Pages with JavaScript

What is the DOM?

The Document Object Model (DOM) is a tree-like representation of your HTML page. JavaScript can read and modify this tree to create dynamic, interactive web pages.

DOM Structure

```
document
  └── <html>
      ├── <head>
      │   └── <title>
      └── <body>
          ├── <h1>
          └── <div>
              └── <p>
```

Selecting Elements

Before you can manipulate an element, you need to select it. JavaScript offers multiple ways:

```javascript
// By ID — returns single element or null
let header = document.getElementById('main-header');

// By CSS selector — returns FIRST matching element
let btn = document.querySelector('.submit-btn');
let link = document.querySelector('a[href]');

// By CSS selector — returns ALL matching elements (NodeList)
let allButtons = document.querySelectorAll('button');
let cards = document.querySelectorAll('.card');

// By tag name
let divs = document.getElementsByTagName('div');

// By class name
let warnings = document.getElementsByClassName('warning');

// Looping over results
cards.forEach(card => {
  console.log(card.textContent);
});
```

Modifying Content

```
let element = document.querySelector('#content');

// Text content (safe — no HTML injection)
element.textContent = 'Hello World';

// HTML content (use carefully to avoid XSS attacks!)
element.innerHTML = '<strong>Bold Text</strong>';

// Attributes
let img = document.querySelector('img');
img.src = 'photo.jpg';
img.alt = 'A beautiful photo';
img.setAttribute('data-id', '42');
console.log(img.getAttribute('src'));  // 'photo.jpg'
img.removeAttribute('alt');

// Form values
let input = document.querySelector('#name-input');
console.log(input.value);  // What user typed
input.value = 'Pre-filled text';
```

Modifying Styles and Classes

```
let box = document.querySelector('.box');

// Direct style (use classes instead when possible)
box.style.backgroundColor = 'blue';
box.style.width = '200px';
box.style.fontSize = '18px';

// CSS Classes (preferred approach)
box.classList.add('active');
box.classList.remove('hidden');
box.classList.toggle('selected');    // Add if not present, remove if present
box.classList.replace('old', 'new'); // Replace class
console.log(box.classList.contains('active'));  // true
```

Creating and Inserting Elements

```
// Create an element
let newCard = document.createElement('div');
newCard.className = 'card';
newCard.textContent = 'New Card Content';
newCard.style.padding = '20px';

// Create with innerHTML
let card = document.createElement('article');
card.innerHTML = `
  <h3 class='card-title'>Product Name</h3>
  <p class='card-price'>$99.99</p>
```

```
  <button class='buy-btn'>Buy Now</button>
`;

// Insert into DOM
let container = document.querySelector('#container');
container.appendChild(newCard);          // Add at end
container.prepend(card);                 // Add at start
container.before(card);                  // Before container
container.after(card);                   // After container
container.insertAdjacentHTML('beforeend', '<p>New paragraph</p>');

// Remove elements
let old = document.querySelector('.old-item');
old.remove();  // Remove from DOM
```

Events

Events let your page respond to user actions like clicks, keystrokes, and mouse movements.

```
let button = document.querySelector('#myBtn');

// addEventListener — preferred method
button.addEventListener('click', function(event) {
  console.log('Button clicked!');
  console.log(event.target);   // The element that was clicked
});

// Arrow function shorthand
button.addEventListener('click', e => {
  e.preventDefault();  // Prevent default behavior (e.g. form submit)
  console.log('Clicked at:', e.clientX, e.clientY);
});

// Keyboard events
document.addEventListener('keydown', e => {
  if (e.key === 'Escape') closeModal();
  if (e.ctrlKey && e.key === 's') saveDocument();
});

// Mouse events
let box = document.querySelector('.hoverable');
box.addEventListener('mouseenter', () => box.classList.add('hovered'));
box.addEventListener('mouseleave', () => box.classList.remove('hovered'));

// Input events
let search = document.querySelector('#search');
search.addEventListener('input', e => {
  console.log('Searching for:', e.target.value);
});
```

Event Delegation

```
// Instead of adding listeners to EACH item, add one to the parent
```

```
let list = document.querySelector('#item-list');

list.addEventListener('click', function(e) {
  // Check what was actually clicked
  if (e.target.classList.contains('delete-btn')) {
    let item = e.target.closest('li');
    item.remove();
    console.log('Item deleted');
  }
  if (e.target.classList.contains('edit-btn')) {
    let item = e.target.closest('li');
    item.contentEditable = true;
    item.focus();
  }
});
// Now handles ALL list items, even ones added dynamically!
```

💡 TIP

Event delegation is crucial for performance. Instead of attaching 100 event listeners to 100 list items, attach ONE listener to the parent container.

📝 Chapter Project: Dynamic To-Do List

```
<!-- HTML needed:
<input id='todo-input' placeholder='Add a task...' />
<button id='add-btn'>Add Task</button>
<ul id='todo-list'></ul>
-->

const input = document.querySelector('#todo-input');
const addBtn = document.querySelector('#add-btn');
const list = document.querySelector('#todo-list');

function addTodo() {
  let text = input.value.trim();
  if (!text) return;

  let li = document.createElement('li');
  li.innerHTML = `
    <span class='todo-text'>${text}</span>
    <button class='done-btn'>✓</button>
    <button class='delete-btn'>✗</button>
  `;
  list.appendChild(li);
  input.value = '';
  input.focus();
}

addBtn.addEventListener('click', addTodo);
input.addEventListener('keypress', e => {
  if (e.key === 'Enter') addTodo();
});
```

```
// Event delegation for done/delete
list.addEventListener('click', e => {
  let li = e.target.closest('li');
  if (!li) return;
  if (e.target.classList.contains('done-btn'))
    li.classList.toggle('completed');
  if (e.target.classList.contains('delete-btn'))
    li.remove();
});
```

CHAPTER 8

Asynchronous JavaScript

Promises, Async/Await & APIs

Understanding Asynchronous Code

JavaScript is single-threaded, but it can handle operations that take time (like network requests) without freezing the page. This is done with callbacks, Promises, and async/await.

```
// Synchronous — each line waits for previous
console.log('1. Start');
// Imagine this takes 3 seconds:
// let data = fetchDataSlowly();  // BLOCKS everything!
console.log('2. End');

// Asynchronous — operations don't block
console.log('1. Start');
setTimeout(() => {
  console.log('2. This runs after 2 seconds');
}, 2000);
console.log('3. This prints IMMEDIATELY');
// Order: 1 → 3 → 2
```

Promises

A Promise represents a value that will be available in the future. It's either pending, fulfilled, or rejected.

```
// Creating a Promise
function fetchUserData(userId) {
  return new Promise((resolve, reject) => {
    // Simulate network request
    setTimeout(() => {
      if (userId > 0) {
        resolve({ id: userId, name: 'Alice', email: 'alice@example.com' });
      } else {
        reject(new Error('Invalid user ID'));
      }
    }, 1000);
  });
}

// Using a Promise with .then() and .catch()
fetchUserData(1)
  .then(user => {
    console.log('User:', user.name);
    return user.email;  // Pass to next .then()
  })
  .then(email => console.log('Email:', email))
  .catch(err => console.error('Error:', err.message))
  .finally(() => console.log('Done — always runs'));
```

```
// Promise.all — run multiple in parallel
Promise.all([fetchUserData(1), fetchUserData(2)])
  .then(([user1, user2]) => {
    console.log(user1.name, user2.name);  // Both available
  });
```

Async/Await — Modern Asynchronous Code

async/await is syntactic sugar over Promises. It makes async code look and read like synchronous code.

```
// async function always returns a Promise
async function getUser(userId) {
  try {
    let user = await fetchUserData(userId);  // Wait here
    console.log('Got user:', user.name);
    return user;
  } catch (err) {
    console.error('Failed:', err.message);
  }
}

// More realistic: chain of async operations
async function loadDashboard(userId) {
  try {
    console.log('Loading...');
    let user = await fetchUserData(userId);
    let posts = await fetchUserPosts(user.id);  // Sequential
    console.log(`${user.name} has ${posts.length} posts`);
  } catch (err) {
    console.error('Dashboard failed:', err.message);
  }
}

// Parallel — much faster!
async function loadDashboardFast(userId) {
  let [user, settings] = await Promise.all([
    fetchUserData(userId),
    fetchUserSettings(userId)  // Both run at same time!
  ]);
  console.log(user.name, settings.theme);
}
```

Fetch API — Making HTTP Requests

The Fetch API is the modern way to make network requests in the browser. It returns a Promise.

```
// GET request — fetch data from a server
async function getJoke() {
  try {
    let response = await fetch('https://api.chucknorris.io/jokes/random');

    if (!response.ok) {  // Always check for HTTP errors
      throw new Error(`HTTP error: ${response.status}`);
```

```
    }

    let data = await response.json();  // Parse JSON body
    console.log(data.value);
  } catch (err) {
    console.error('Fetch failed:', err.message);
  }
}
getJoke();

// POST request — send data to a server
async function createPost(postData) {
  let response = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(postData)
  });
  let result = await response.json();
  console.log('Created:', result);
}

createPost({ title: 'Hello', body: 'World', userId: 1 });
```

🚀 **TRY IT — REAL WORLD EXAMPLE**

Real-World: Every modern web app uses fetch — React/Vue apps fetch data from REST APIs, payment forms POST credit card data securely, and dashboards fetch real-time data.

Error Handling

```
// try/catch/finally
function divide(a, b) {
  if (b === 0) throw new Error('Division by zero!');
  return a / b;
}

try {
  console.log(divide(10, 2));   // 5
  console.log(divide(10, 0));   // throws Error
} catch (err) {
  console.error(`Error: ${err.message}`);  // Error: Division by zero!
} finally {
  console.log('This always runs');
}

// Custom Error classes
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}
```

```
function validateAge(age) {
  if (typeof age !== 'number') throw new ValidationError('age', 'Age must be a
number');
  if (age < 0 || age > 150)    throw new ValidationError('age', 'Age out of
range');
  return true;
}

try {
  validateAge('twenty');
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(`Validation failed on '${err.field}': ${err.message}`);
  }
}
```

LocalStorage & SessionStorage

```
// localStorage persists until explicitly cleared
// sessionStorage clears when browser tab closes

// Save data
localStorage.setItem('username', 'alice');
localStorage.setItem('theme', 'dark');

// Store objects (must stringify/parse)
let userPrefs = { theme: 'dark', fontSize: 16, language: 'en' };
localStorage.setItem('prefs', JSON.stringify(userPrefs));

// Read data
let name = localStorage.getItem('username');   // 'alice'
let prefs = JSON.parse(localStorage.getItem('prefs'));
console.log(prefs.theme);  // 'dark'

// Delete
localStorage.removeItem('username');
localStorage.clear();  // Remove ALL items

// Real-world pattern: save app state
function saveCart(cart) {
  localStorage.setItem('cart', JSON.stringify(cart));
}
function loadCart() {
  return JSON.parse(localStorage.getItem('cart')) || [];
}
let cart = loadCart();  // Empty array if nothing saved
```

📝 Chapter Project: Weather App (using a public API)

```
// Uses the free Open-Meteo API (no API key needed)
async function getWeather(lat, lon) {
  const url = `https://api.open-meteo.com/v1/forecast` +
```

```
    `?latitude=${lat}&longitude=${lon}` +
    `&current_weather=true&temperature_unit=fahrenheit`;

  try {
    let response = await fetch(url);
    if (!response.ok) throw new Error('Network error');
    let data = await response.json();

    let { temperature, windspeed, weathercode } = data.current_weather;
    let condition = weathercode === 0 ? 'Clear' :
                    weathercode <= 3 ? 'Partly Cloudy' :
                    weathercode <= 67 ? 'Rainy' : 'Stormy';

    console.log(`Temperature: ${temperature}°F`);
    console.log(`Wind Speed: ${windspeed} mph`);
    console.log(`Condition: ${condition}`);

    // Cache result
    localStorage.setItem('lastWeather', JSON.stringify({
      temperature, windspeed, condition,
      timestamp: new Date().toISOString()
    }));
  } catch (err) {
    console.error('Could not fetch weather:', err.message);
    // Fall back to cached data
    let cached = JSON.parse(localStorage.getItem('lastWeather'));
    if (cached) {
      console.log('Showing cached weather from:', cached.timestamp);
      console.log(`Temperature: ${cached.temperature}°F`);
    }
  }
}

// New York City coordinates
getWeather(40.7128, -74.0060);
```

CHAPTER 9

Advanced JavaScript

Modules, Patterns & Best Practices

ES6 Modules

Modules let you split your code into separate files and import/export functionality between them. Modern JavaScript development always uses modules.

```
// === math.js (module file) ===

// Named exports — export specific items
export const PI = 3.14159;

export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// Default export — one per file
export default class Calculator {
  add(a, b) { return a + b; }
  subtract(a, b) { return a - b; }
}

// === app.js (consuming file) ===

// Import named exports
import { PI, add, subtract } from './math.js';

// Import default
import Calculator from './math.js';

// Import everything as namespace
import * as MathLib from './math.js';

// Rename on import
import { add as sum } from './math.js';

console.log(add(2, 3));         // 5
console.log(MathLib.PI);        // 3.14159

// In HTML: <script type='module' src='app.js'></script>
```

Destructuring & Advanced Patterns

```
// Optional chaining — safely access nested properties
let user = {
  profile: {
    address: { city: 'NYC' }
  }
```

```
};

// Old way (verbose):
let city1 = user && user.profile && user.profile.address &&
user.profile.address.city;

// New way with ?.
let city2 = user?.profile?.address?.city;  // 'NYC'
let zip = user?.profile?.address?.zip;     // undefined (no error!)

// Nullish coalescing ?? — use default only if null/undefined
let username = null;
let display = username ?? 'Anonymous';  // 'Anonymous'
let count = 0;
let displayCount = count ?? 10;  // 0 (not 10 — 0 is not null!)
let displayCount2 = count || 10; // 10 (|| treats 0 as falsy)

// Logical assignment operators
let a = null;
a ??= 'default';   // a = 'default' (assign if null/undefined)
let b = '';
b ||= 'fallback';  // b = 'fallback' (assign if falsy)
let c = 5;
c &&= c * 2;       // c = 10 (assign if truthy)
```

Iterators and Generators

```
// Generator function — produces a sequence of values
function* range(start, end, step = 1) {
  for (let i = start; i <= end; i += step) {
    yield i;  // Pause and return value
  }
}

for (let num of range(1, 10, 2)) {
  process.stdout.write(num + ' ');  // 1 3 5 7 9
}

// Infinite generator — IDs
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

let nextId = idGenerator();
console.log(nextId.next().value);  // 1
console.log(nextId.next().value);  // 2
console.log(nextId.next().value);  // 3
```

Proxy and Reflect

```
// Proxy intercepts operations on objects
let handler = {
  get(target, prop) {
    console.log(`Getting ${prop}`);
    return prop in target ? target[prop] : `Property '${prop}' not found`;
  },
  set(target, prop, value) {
    if (typeof value !== 'number') {
      throw new TypeError(`${prop} must be a number`);
    }
    target[prop] = value;
    return true;
  }
};

let product = new Proxy({}, handler);
product.price = 99;       // OK
console.log(product.price);  // Getting price → 99
// product.price = 'free';  // TypeError!
```

Design Patterns

Module Pattern

```
// Encapsulate state and expose public API
const ShoppingCart = (() => {
  // Private state
  let items = [];
  let discount = 0;

  // Public API
  return {
    add(item) {
      items.push(item);
      console.log(`Added: ${item.name}`);
    },
    remove(name) {
      items = items.filter(i => i.name !== name);
    },
    setDiscount(pct) { discount = pct; },
    get total() {
      let subtotal = items.reduce((sum, i) => sum + i.price, 0);
      return subtotal * (1 - discount / 100);
    },
    get count() { return items.length; }
  };
})();

ShoppingCart.add({ name: 'Shirt', price: 29.99 });
ShoppingCart.add({ name: 'Pants', price: 59.99 });
ShoppingCart.setDiscount(10);
console.log(`Total: $${ShoppingCart.total.toFixed(2)}`);  // $80.98
```

Observer Pattern

```
// Components can subscribe to and publish events
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(event, listener) {
    if (!this.events[event]) this.events[event] = [];
    this.events[event].push(listener);
    return this;  // For chaining
  }

  emit(event, ...args) {
    if (this.events[event]) {
      this.events[event].forEach(listener => listener(...args));
    }
  }

  off(event, listener) {
    this.events[event] = (this.events[event] || [])
      .filter(l => l !== listener);
  }
}

let store = new EventEmitter();

store.on('userLoggedIn', user => console.log(`Welcome, ${user.name}!`));
store.on('userLoggedIn', user => console.log(`Sending welcome email to $
{user.email}`));

store.emit('userLoggedIn', { name: 'Alice', email: 'alice@example.com' });
// Welcome, Alice!
// Sending welcome email to alice@example.com
```

Performance Best Practices

🚀 Performance Tips

1. Batch DOM updates — read all DOM, then write all DOM (avoid interleaving)
2. Use DocumentFragment for multiple insertions
3. Debounce/throttle frequent events (scroll, resize, input)
4. Cache DOM queries — don't query inside loops
5. Use const over let when possible — signals intent
6. Prefer array methods (map/filter) over manual loops when readable
7. Avoid global variables — use modules or closures
8. Use === instead of == to prevent unexpected type coercions

```
// Debounce — limit how often a function runs
function debounce(fn, delay) {
  let timer;
  return (...args) => {
```

```
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// Only runs 300ms AFTER user stops typing
let search = debounce(query => {
  console.log(`Searching for: ${query}`);
}, 300);

// DocumentFragment for batch insertions
let fragment = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
  let item = document.createElement('li');
  item.textContent = `Item ${i}`;
  fragment.appendChild(item);  // Add to fragment, NOT DOM
}
document.querySelector('ul').appendChild(fragment);  // ONE DOM update
```

CHAPTER 10

Putting It All Together

Real-World Projects

Project 1: Shopping Cart Application

A complete shopping cart with add/remove, quantity updates, discounts, and local storage persistence.

```
// Complete Shopping Cart Application
class Cart {
  constructor() {
    this.items = JSON.parse(localStorage.getItem('cart')) || [];
    this.discountCode = null;
    this.discounts = { SAVE10: 10, SAVE20: 20, HALFOFF: 50 };
  }

  addItem(product, qty = 1) {
    let existing = this.items.find(i => i.id === product.id);
    if (existing) {
      existing.qty += qty;
    } else {
      this.items.push({ ...product, qty });
    }
    this.save();
    return this;
  }

  removeItem(productId) {
    this.items = this.items.filter(i => i.id !== productId);
    this.save();
    return this;
  }

  updateQty(productId, qty) {
    if (qty <= 0) return this.removeItem(productId);
    let item = this.items.find(i => i.id === productId);
    if (item) item.qty = qty;
    this.save();
    return this;
  }

  applyDiscount(code) {
    if (this.discounts[code]) {
      this.discountCode = code;
      console.log(`Discount applied: ${this.discounts[code]}% off`);
    } else {
      console.log('Invalid discount code');
    }
    return this;
  }

  get subtotal() {
    return this.items.reduce((sum, i) => sum + (i.price * i.qty), 0);
```

```
  }

  get discountAmount() {
    if (!this.discountCode) return 0;
    return this.subtotal * (this.discounts[this.discountCode] / 100);
  }

  get total() {
    return this.subtotal - this.discountAmount;
  }

  save() {
    localStorage.setItem('cart', JSON.stringify(this.items));
  }

  printReceipt() {
    console.log('='.repeat(40));
    console.log('RECEIPT');
    console.log('='.repeat(40));
    this.items.forEach(i => {
      let line = `${i.name} x${i.qty}`.padEnd(25);
      console.log(`${line} $${(i.price * i.qty).toFixed(2)}`);
    });
    console.log('-'.repeat(40));
    console.log(`Subtotal:`.padEnd(25) + ` $${this.subtotal.toFixed(2)}`);
    if (this.discountCode) {
      console.log(`Discount (${this.discountCode}):`.padEnd(25) + `-$$
{this.discountAmount.toFixed(2)}`);
    }
    console.log(`TOTAL:`.padEnd(25) + ` $${this.total.toFixed(2)}`);
    console.log('='.repeat(40));
  }
}

// Test it
let cart = new Cart();
cart.addItem({ id: 1, name: 'JavaScript Book', price: 39.99 })
    .addItem({ id: 2, name: 'Code Editor License', price: 79.99 })
    .addItem({ id: 1, name: 'JavaScript Book', price: 39.99 }, 2)
    .applyDiscount('SAVE10');

cart.printReceipt();
```

Project 2: Quiz Application

```
class QuizApp {
  constructor(questions) {
    this.questions = questions;
    this.currentIndex = 0;
    this.score = 0;
    this.answers = [];
    this.startTime = Date.now();
  }

  get currentQuestion() {
```

```
    return this.questions[this.currentIndex];
  }

  get isFinished() {
    return this.currentIndex >= this.questions.length;
  }

  answer(choice) {
    if (this.isFinished) return;
    let q = this.currentQuestion;
    let correct = choice === q.answer;
    if (correct) this.score++;
    this.answers.push({ question: q.text, choice, correct, expected: q.answer });
    this.currentIndex++;
  }

  get results() {
    let timeTaken = ((Date.now() - this.startTime) / 1000).toFixed(1);
    let pct = ((this.score / this.questions.length) * 100).toFixed(0);
    let grade = pct >= 90 ? 'A' : pct >= 80 ? 'B' : pct >= 70 ? 'C' : pct >= 60 ?
'D' : 'F';
    return { score: this.score, total: this.questions.length, pct, grade, timeTaken
};
  }

  printReport() {
    let r = this.results;
    console.log(`\n📊 QUIZ RESULTS`);
    console.log(`Score: ${r.score}/${r.total} (${r.pct}%) — Grade: ${r.grade}`);
    console.log(`Time: ${r.timeTaken}s\n`);
    this.answers.forEach((a, i) => {
      let icon = a.correct ? '✅' : '❌';
      console.log(`${icon} Q${i+1}: ${a.question}`);
      if (!a.correct) console.log(`   Your answer: ${a.choice} | Correct: $
{a.expected}`);
    });
  }
}

let quiz = new QuizApp([
  { text: 'What does DOM stand for?', options: ['A','B','C','D'],
    answer: 'Document Object Model' },
  { text: 'Which method adds to end of array?',
    answer: 'push()' },
  { text: 'What does === check?',
    answer: 'Value and type equality' }
]);

quiz.answer('Document Object Model');
quiz.answer('push()');
quiz.answer('Value only');  // Wrong!
quiz.printReport();
```

Project 3: Real-Time Search Filter

```
<!--
```

```
<input id='search' placeholder='Search products...' />
<select id='category-filter'>
  <option value='all'>All Categories</option>
  <option value='electronics'>Electronics</option>
  <option value='clothing'>Clothing</option>
</select>
<div id='results'></div>
-->

const products = [
  { id: 1, name: 'Laptop', category: 'electronics', price: 999 },
  { id: 2, name: 'T-Shirt', category: 'clothing', price: 25 },
  { id: 3, name: 'Headphones', category: 'electronics', price: 149 },
  { id: 4, name: 'Jeans', category: 'clothing', price: 65 },
  { id: 5, name: 'Keyboard', category: 'electronics', price: 89 },
];

function renderProducts(items) {
  let container = document.querySelector('#results');
  if (items.length === 0) {
    container.innerHTML = '<p>No products found.</p>';
    return;
  }
  container.innerHTML = items.map(p => `
    <div class='product-card'>
      <h3>${p.name}</h3>
      <span class='badge'>${p.category}</span>
      <span class='price'>$${p.price}</span>
    </div>
  `).join('');
}

function filterProducts() {
  let query = document.querySelector('#search').value.toLowerCase().trim();
  let category = document.querySelector('#category-filter').value;

  let filtered = products.filter(p => {
    let matchesSearch = p.name.toLowerCase().includes(query);
    let matchesCategory = category === 'all' || p.category === category;
    return matchesSearch && matchesCategory;
  });

  renderProducts(filtered);
}

// Debounce search for performance
let debouncedFilter = debounce(filterProducts, 250);

document.querySelector('#search').addEventListener('input', debouncedFilter);
document.querySelector('#category-filter').addEventListener('change',
filterProducts);

renderProducts(products);  // Initial render
```

CHAPTER 11

HTML5 APIs & Next Steps

Canvas, Geolocation & The Road Ahead

HTML5 Canvas

Canvas lets you draw graphics, animations, and games directly in the browser using JavaScript.

```
<!-- In HTML: <canvas id='myCanvas' width='600' height='400'></canvas> -->

let canvas = document.querySelector('#myCanvas');
let ctx = canvas.getContext('2d');  // Get 2D drawing context

// Drawing shapes
ctx.fillStyle = '#1A56DB';
ctx.fillRect(50, 50, 200, 100);  // x, y, width, height

ctx.strokeStyle = '#047857';
ctx.lineWidth = 3;
ctx.strokeRect(300, 50, 200, 100);

// Drawing circles
ctx.beginPath();
ctx.arc(150, 250, 60, 0, Math.PI * 2);  // x, y, radius, startAngle, endAngle
ctx.fillStyle = 'rgba(255, 100, 0, 0.7)';
ctx.fill();

// Drawing text
ctx.fillStyle = '#111827';
ctx.font = 'bold 24px Arial';
ctx.fillText('Hello Canvas!', 50, 370);

// Drawing a line
ctx.beginPath();
ctx.moveTo(400, 200);
ctx.lineTo(550, 350);
ctx.strokeStyle = 'purple';
ctx.lineWidth = 4;
ctx.stroke();
```

Canvas Animation

```
// Bouncing ball animation
let canvas = document.querySelector('#myCanvas');
let ctx = canvas.getContext('2d');

let ball = {
  x: 300, y: 200,
  radius: 20,
  dx: 4, dy: -3,  // Velocity
```

```
  color: '#1A56DB'
};

function animate() {
  // Clear previous frame
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  // Draw ball
  ctx.beginPath();
  ctx.arc(ball.x, ball.y, ball.radius, 0, Math.PI * 2);
  ctx.fillStyle = ball.color;
  ctx.fill();

  // Move ball
  ball.x += ball.dx;
  ball.y += ball.dy;

  // Bounce off walls
  if (ball.x + ball.radius > canvas.width || ball.x - ball.radius < 0)
    ball.dx = -ball.dx;
  if (ball.y + ball.radius > canvas.height || ball.y - ball.radius < 0)
    ball.dy = -ball.dy;

  requestAnimationFrame(animate);  // ~60fps
}

animate();  // Start the loop
```

Geolocation API

```
// Get user's location (requires permission)
function getLocation() {
  if (!navigator.geolocation) {
    console.log('Geolocation not supported');
    return;
  }

  navigator.geolocation.getCurrentPosition(
    (position) => {
      let { latitude, longitude, accuracy } = position.coords;
      console.log(`Latitude: ${latitude.toFixed(4)}`);
      console.log(`Longitude: ${longitude.toFixed(4)}`);
      console.log(`Accuracy: ±${accuracy.toFixed(0)} meters`);

      // Use coordinates to fetch weather
      getWeather(latitude, longitude);
    },
    (error) => {
      switch (error.code) {
        case 1: console.log('Permission denied'); break;
        case 2: console.log('Position unavailable'); break;
        case 3: console.log('Timeout'); break;
      }
    },
    { timeout: 10000, enableHighAccuracy: true }
```

```
  );
}
```

File API

```
<!-- <input type='file' id='fileInput' accept='.txt,.json' /> -->

document.querySelector('#fileInput').addEventListener('change', function(e) {
  let file = e.target.files[0];
  if (!file) return;

  console.log(`Name: ${file.name}`);
  console.log(`Size: ${(file.size / 1024).toFixed(2)} KB`);
  console.log(`Type: ${file.type}`);

  let reader = new FileReader();

  reader.onload = function(e) {
    let content = e.target.result;

    if (file.type === 'application/json') {
      let data = JSON.parse(content);
      console.log('Parsed JSON:', data);
    } else {
      console.log('File content:', content.substring(0, 200));
    }
  };

  reader.onerror = () => console.error('Error reading file');
  reader.readAsText(file);
});
```

Intersection Observer — Lazy Loading

```
// Detect when elements enter the viewport
let observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      let img = entry.target;
      img.src = img.dataset.src;  // Load the real image
      img.classList.add('loaded');
      observer.unobserve(img);   // Stop watching
    }
  });
}, {
  threshold: 0.1,    // 10% visible triggers callback
  rootMargin: '0px 0px 200px 0px'  // Preload 200px before visible
});

// Observe all lazy images
document.querySelectorAll('img[data-src]').forEach(img => {
  observer.observe(img);
```

```
});
```

Web APIs Quick Reference

API	Use Case	Key Method
`Fetch`	`HTTP requests`	`fetch(url, options)`
localStorage	Persist data	setItem / getItem
Geolocation	GPS coordinates	getCurrentPosition()
Canvas	2D drawing	getContext("2d")
File	Read local files	FileReader.readAsText()
Clipboard	Copy/paste	navigator.clipboard.writeText()
Notification	Push notifications	new Notification(text)
WebSocket	Real-time comms	new WebSocket(url)
Intersection Observer	Lazy load / scroll effects	new IntersectionObserver(cb)
ResizeObserver	Element resize events	new ResizeObserver(cb)

What's Next — The JavaScript Ecosystem

You've learned the core of JavaScript. Here's your roadmap for further growth:

🗺️ Learning Roadmap

FRONTEND FRAMEWORKS
→ React.js — most popular, component-based UI (Meta)
→ Vue.js — approachable, great documentation
→ Angular — full framework by Google

BACKEND (Node.js)
→ Express.js — minimal web server framework
→ NestJS — opinionated, TypeScript-first
→ Next.js — React + server-side rendering

LANGUAGE EXTENSIONS
→ TypeScript — JavaScript with static typing

TOOLS TO LEARN
→ npm / yarn — package managers
→ Webpack / Vite — bundlers
→ Git — version control
→ Chrome DevTools — debugging

📝 Final Project: Full Mini-Application

Build a complete task manager with categories, deadlines, filtering, and local storage persistence — combining everything from all chapters.

```
class TaskManager {
  constructor() {
    this.tasks = JSON.parse(localStorage.getItem('tasks')) || [];
    this.nextId = parseInt(localStorage.getItem('nextId')) || 1;
  }

  addTask({ title, category = 'General', priority = 'medium', due = null }) {
    if (!title.trim()) throw new ValidationError('title', 'Title required');
    let task = {
      id: this.nextId++,
      title: title.trim(),
      category, priority, due,
      done: false,
      created: new Date().toISOString()
    };
    this.tasks.push(task);
    this.save();
    return task;
  }

  complete(id) {
    let task = this.tasks.find(t => t.id === id);
    if (task) { task.done = !task.done; this.save(); }
    return task;
  }

  delete(id) {
    this.tasks = this.tasks.filter(t => t.id !== id);
    this.save();
  }

  filter({ category, priority, done, search } = {}) {
    return this.tasks.filter(t => {
      if (category && t.category !== category) return false;
      if (priority && t.priority !== priority) return false;
      if (done !== undefined && t.done !== done) return false;
      if (search && !t.title.toLowerCase().includes(search.toLowerCase())) return
false;
      return true;
    }).sort((a, b) => {
      const P = { high: 0, medium: 1, low: 2 };
      return P[a.priority] - P[b.priority];
    });
  }

  get stats() {
    let total = this.tasks.length;
    let done = this.tasks.filter(t => t.done).length;
    let overdue = this.tasks.filter(t => t.due && new Date(t.due) < new Date() && !
t.done).length;
    return { total, done, pending: total - done, overdue };
  }
```

```
  save() {
    localStorage.setItem('tasks', JSON.stringify(this.tasks));
    localStorage.setItem('nextId', this.nextId);
  }
}

// Usage
let manager = new TaskManager();
manager.addTask({ title: 'Learn JavaScript', category: 'Study', priority: 'high',
due: '2024-12-31' });
manager.addTask({ title: 'Build a project', category: 'Study', priority: 'high' });
manager.addTask({ title: 'Buy groceries', category: 'Personal', priority: 'low' });

let highPriority = manager.filter({ priority: 'high' });
console.log('High priority tasks:', highPriority.map(t => t.title));

manager.complete(1);
let pending = manager.filter({ done: false });
console.log('Pending:', pending.length);

console.log('Stats:', manager.stats);
```

CHAPTER 12

Quick Reference

JavaScript Cheatsheet

Essential JavaScript Cheatsheet

Variable Declaration

```
const PI = 3.14;           // Immutable — use for values that don't change
let counter = 0;           // Block-scoped — use for everything else
// var name = 'old';       // Avoid — function-scoped, hoisted
```

Template Literals

```
let name = 'World';
let msg = `Hello, ${name}!`;          // Hello, World!
let multiLine = `
  Line 1
  Line 2
`;  // Preserves line breaks
```

Destructuring Shortcuts

```
let [a, b, ...rest] = [1, 2, 3, 4, 5]; // a=1, b=2, rest=[3,4,5]
let { x, y = 10 } = { x: 5 };          // x=5, y=10 (default)
let { name: alias } = { name: 'Alice' }; // Rename: alias='Alice'
```

Async/Await Pattern

```
async function fetchData(url) {
  try {
    let res = await fetch(url);
    if (!res.ok) throw new Error(res.status);
    return await res.json();
  } catch (err) {
    console.error(err);
    return null;
  }
}
```

Array Methods Reference

Method	Returns	Description
`map(fn)`	New array	Transform every element
`filter(fn)`	New array	Keep elements where fn returns true

`reduce(fn, init)`	Single value	Accumulate array into one value
`find(fn)`	Element or undefined	First element where fn is true
`findIndex(fn)`	Number	Index of first match, or -1
`some(fn)`	Boolean	True if ANY element matches
`every(fn)`	Boolean	True if ALL elements match
`flat(depth)`	New array	Flatten nested arrays
`flatMap(fn)`	New array	map() then flat(1)
`includes(val)`	Boolean	Check if value exists
`indexOf(val)`	Number	Index of value, or -1
`slice(start,end)`	New array	Extract portion (non-destructive)
`splice(i,n)`	Removed items	Remove/insert at index (destructive)
`sort(fn)`	Same array	Sort in-place (pass comparator for numbers)
`reverse()`	Same array	Reverse in-place
`join(sep)`	String	Combine array into string
`forEach(fn)`	undefined	Execute fn for each element (no return)
`push(val)`	New length	Add to end
`pop()`	Removed item	Remove from end
`unshift(val)`	New length	Add to beginning
`shift()`	Removed item	Remove from beginning

Common DOM Methods

Method	Description
`querySelector(sel)`	First matching element
`querySelectorAll(sel)`	All matching elements (NodeList)
`getElementById(id)`	Element by ID
`createElement(tag)`	Create new element
`appendChild(el)`	Add child at end
`removeChild(el) / el.remove()`	Delete element from DOM
`addEventListener(evt, fn)`	Attach event handler
`removeEventListener(evt, fn)`	Detach event handler
`classList.add/remove/toggle`	Manipulate CSS classes
`style.property = value`	Set inline style
`textContent`	Get/set text content (safe)
`innerHTML`	Get/set HTML content (XSS risk!)
`getAttribute(name)`	Read attribute value
`setAttribute(name, val)`	Set attribute value
`dataset.name`	Access data-* attributes

Event Types

```
// Mouse: click, dblclick, mousedown, mouseup, mousemove, mouseenter, mouseleave
// Keyboard: keydown, keyup, keypress
// Form: submit, change, input, focus, blur, reset
// Document: DOMContentLoaded, load, resize, scroll
// Touch: touchstart, touchmove, touchend
// Custom: dispatchEvent(new CustomEvent('myEvent', { detail: data }))
```

Common Interview Questions

🎯 Key Concepts to Master

1. What is the difference between == and ===?
   → == coerces types; === checks value AND type (always use ===)

2. What is a closure?
   → A function that retains access to its outer scope variables

3. What is the event loop?
   → JS runs call stack synchronously; async callbacks go to callback queue → event loop picks up when stack is empty

4. What is the difference between null and undefined?
   → undefined: variable declared but not assigned; null: intentionally empty value

5. What is hoisting?
   → function declarations and var declarations are moved to top of scope at parse time

6. let vs const vs var?
   → var: function-scoped, hoisted; let/const: block-scoped; const: cannot be reassigned

7. What is prototypal inheritance?
   → Objects inherit from other objects via prototype chain. classes are syntactic sugar.

8. Explain async/await vs Promises
   → async/await is syntactic sugar for .then() chains; both handle async operations

🎓 Congratulations!

You've completed JavaScript from Beginner to Professional!

You now know:
- ✓ Variables, data types, and operators
- ✓ Arrays, objects, and destructuring
- ✓ Control flow — if/else, switch, all loop types
- ✓ Functions — closures, callbacks, arrow functions, recursion
- ✓ Classes and object-oriented programming
- ✓ Built-in string, number, array, and date methods

- ✓ DOM manipulation and event handling
- ✓ Async JavaScript — Promises, async/await, Fetch API
- ✓ Modern JS — modules, optional chaining, nullish coalescing
- ✓ HTML5 APIs — Canvas, Geolocation, File, Storage

Next steps: Build real projects, learn a framework (React/Vue), contribute to open source, and never stop coding!